

BrokerVerse

Out-of-the-box insurance broking platform

Solution Architecture · Document 07 of 11

High Availability and Resiliency

Product	BrokerVerse
Document	07 High Availability and Resiliency
Version	1.0
Date	29 September 2026
Prepared by	iorta TechNXT
Classification	Confidential: for the client, iorta TechNXT and authorised partners

Document control

Document	BrokerVerse Solution Architecture: 07 High Availability and Resiliency
Document ID	BV-SA-07
Version	1.0
Status	Issued for review
Date	29 September 2026
Prepared by	iorta TechNXT
Source baseline	BrokerVerse source repository, branch brokerverse-platform (backend/, brokerverse/, docker-compose.yml, render.yaml, docs/)

Version history

Version	Date	Author	Change
1.0	29 September 2026	iorta TechNXT	Initial issue

Review and approval

Role	Name	Signature	Date
Solution architect			
DevOps / infrastructure lead			
Information security			
Business owner			

Conventions

- Statements of fact are derived from the code, configuration and database of the baseline above (file paths are given relative to the repository). Figures measured on the local test system are labelled **test data** or **indicative**.
- Targets, sizes, retention periods and thresholds that are not facts of the code are labelled **Recommended / to be confirmed by the business and DevOps**.
- Items marked **Gap** are defects or missing capabilities found while preparing this document.

Contents

- 1 Introduction..... 4
 - 1.1 Purpose and scope..... 4
- 2 Application properties that enable high availability..... 5
 - 2.1 Stateless API..... 5
 - 2.2 Health endpoints..... 5
 - 2.3 Start-up and graceful shutdown..... 5
 - 2.4 Scheduled jobs on several instances..... 5
 - 2.5 Transactions, retries and idempotency..... 6
- 3 Recommended highly available deployment..... 7
- 4 Single points of failure and mitigations..... 8
- 5 Known limitations..... 9
- 6 Related documents..... 10
- 7 Glossary..... 11

1 Introduction

1.1 Purpose and scope

This document describes how BrokerVerse stays available when components fail: the properties of the application that allow redundancy (stateless API, health endpoints, graceful shutdown, single execution of scheduled jobs), the recommended highly available deployment in AWS, the single points of failure and their mitigations, and the known limitations. Recovery objectives are in document 08, backups and restores in document 09.

Recommended / to be confirmed by the business and DevOps: The AWS topology in this document (Multi-AZ RDS, EFS, two or more API tasks across two Availability Zones, CloudFront) is the recommended target derived from `docs/GO_LIVE_CHECKLIST.md`; it is not yet provisioned in the baseline.

2 Application properties that enable high availability

2.1 Stateless API

The API keeps no user session in memory: every request carries its access token, and all state is in PostgreSQL (records, refresh tokens, token versions, job runs, outbox) or in the file store (`UPLOAD_DIR`). Any instance can serve any request, so instances can be added, removed or replaced behind a load balancer without sticky sessions. Two pieces of state are per process and therefore not shared: the settings cache and the rate-limit counters (see chapter 6).

2.2 Health endpoints

Endpoint	Use	Behaviour (<code>backend/src/lib/health.js</code>)
GET <code>/api/health</code>	Readiness: load balancer target health, Docker HEALTHCHECK, Render healthCheckPath	200 { status: "ok", ready: true, database: { reachable, latencyMs }, pendingMigrations: 0, uptimeSeconds } when start-up (migrations and seed) has finished, PostgreSQL answers <code>SELECT 1</code> within <code>HEALTH_DB_TIMEOUT_MS</code> (2,000 ms) and no migration is pending; otherwise 503 with status <code>starting</code> or <code>unavailable</code> . Also 503 as soon as shutdown begins, so traffic drains.
GET <code>/api/health/live</code>	Liveness: container restart probe	200 while the process serves HTTP; no database check, so a database outage does not cause restart loops
GET <code>/api/version</code>	Monitoring	Name, version, commit (<code>GIT_COMMIT</code>), start time, uptime, Node version, environment, database reachability, pending migrations

The health endpoints need no sign-in, are not rate limited and are not written to the access log.

2.3 Start-up and graceful shutdown

1. The server checks the production configuration and exits with a clear log line if it is unsafe.
2. It marks itself not ready, applies pending migrations and the idempotent seed, starts HTTP, starts the scheduler and then marks itself ready.
3. On `SIGTERM` or `SIGINT` (ECS task stop, rolling deployment, scale-in) it marks itself not ready (readiness 503), stops the scheduler, stops accepting new connections, lets in-flight requests finish, closes idle keep-alive connections and the database pool, and exits. A forced exit happens after `SHUTDOWN_TIMEOUT_MS` (25 s by default).

Recommended / to be confirmed by the business and DevOps: Set the ECS `stopTimeout` to 30 s (above the 25 s shutdown timeout) and the ALB deregistration delay to 30 s, so in-flight requests complete during deployments.

2.4 Scheduled jobs on several instances

Every job run takes a PostgreSQL session advisory lock keyed on the job code (`pg_try_advisory_lock`) on a dedicated connection; an instance that cannot take the lock skips the run. A scheduled run is also skipped if another instance already started a scheduled run of the same job in the same minute. Therefore the scheduler can run on all instances (`SCHEDULER_ENABLED=true`) without double execution, and a failed instance does not stop the jobs as long as one instance with the scheduler is running. Advisory locks are released automatically if the holding connection or instance dies.

2.5 Transactions, retries and idempotency

Aspect	Behaviour
Atomic business operations	Multi-table operations (quotation to policy with bill, journals, collection item and commission; receipts and applications; vouchers; debit notes; remittance decisions) run in one database transaction (<code>withTransaction</code>); a failure rolls back everything, so a retry does not leave half-created documents.
Concurrency guards	Rows are locked <code>FOR UPDATE</code> where a double action would be harmful (for example converting a quotation: a second concurrent request fails with "converted by another request"); unique indexes prevent duplicates (one open renewal per policy, one pending approval per entity, one commission line per policy, referrer and chain position, unique document numbers).
Client retries	The front end retries a request once after refreshing an expired access token. Other failed requests are not retried automatically; the user repeats the action. POST requests do not carry idempotency keys, so a timed-out save that actually succeeded can be repeated by the user; the uniqueness rules above prevent the most harmful duplicates.
E-mail	Queued in the database and sent by the outbox job with up to 5 attempts; an SMTP outage delays e-mail but does not fail business transactions.
Renewal notice queue	<code>job_queue</code> items are retried by the <code>renewal-queue</code> job; stale items are picked up again.
Database failover	During an RDS Multi-AZ failover (typically 60 to 120 s) open connections fail; requests in that window return 500 or 503 and the pool reconnects automatically afterwards. Readiness answers 503 while the database is unreachable, but the liveness probe keeps the tasks running.

3 Recommended highly available deployment

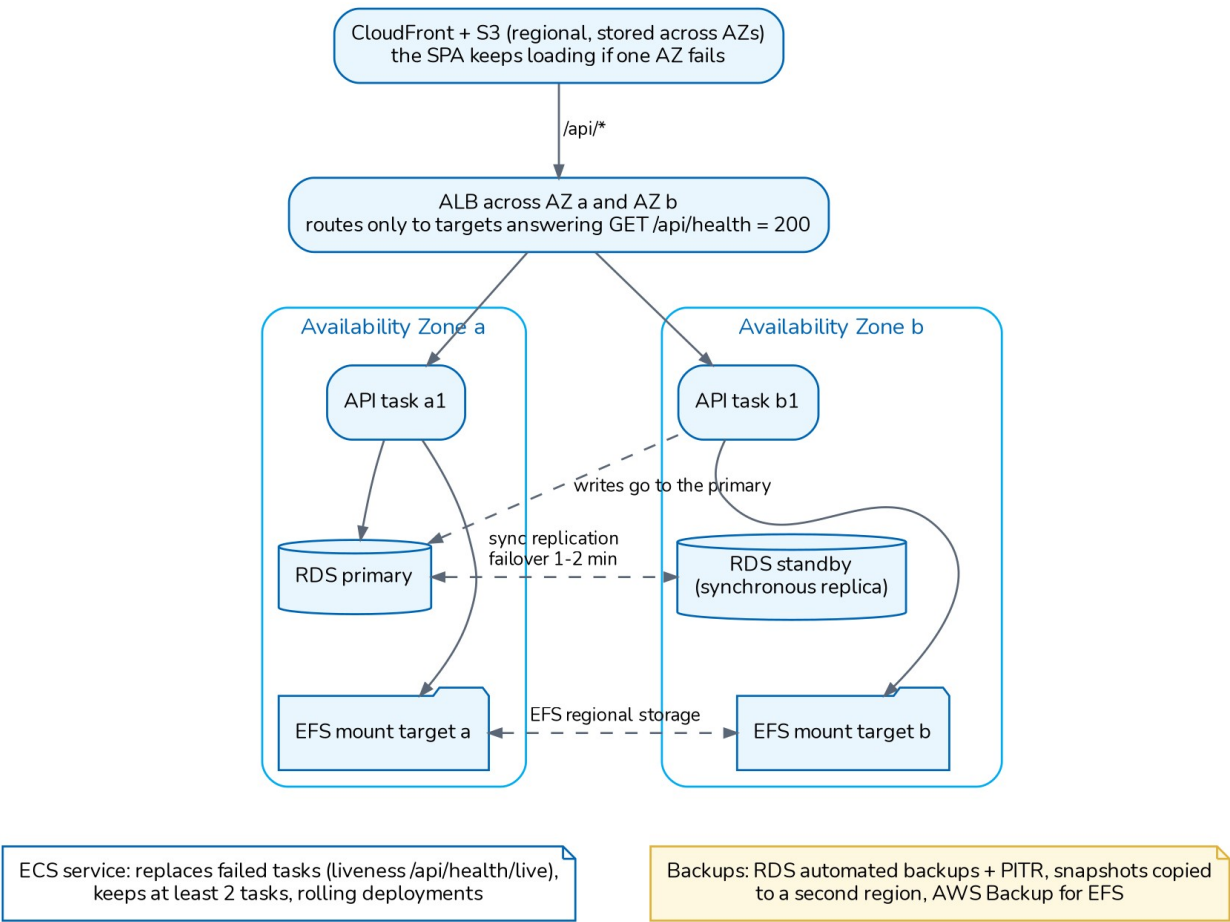


Figure 1. High-availability topology and failure domains (recommended)

Tier	Redundancy	Behaviour on failure
DNS and edge	Route 53, CloudFront (global edge), optional WAF	Managed by AWS; no single instance
Front end	S3 (regional, stores objects across AZs) with versioning	The SPA remains available during API outages (screens show errors until the API returns)
Load balancer	ALB across two AZs	Managed; routes only to healthy targets (/api/health)
API	ECS service with at least two tasks spread over two AZs; rolling deployment with minimum healthy 100 %, maximum 200 %	A failed task is replaced automatically (liveness); a failed AZ leaves the tasks of the other AZ serving, the service scheduler starts replacements there
Database	RDS PostgreSQL Multi-AZ (synchronous standby)	Automatic failover to the standby in the other AZ, same endpoint name; applications reconnect
File store	EFS (regional storage class, mount targets in each AZ)	Files remain available from the other AZ
Secrets	Secrets Manager / SSM (regional)	Read at task start
E-mail	SMTP relay (for example SES)	Outbox retries; e-mail delayed, not lost

4 Single points of failure and mitigations

Single point of failure	Current state (baseline)	Mitigation
Database	One PostgreSQL instance (Docker volume or Render basic plan)	RDS Multi-AZ in production; PITR and snapshots (document 09)
File store	Local directory / Docker volume / Render 1 GB disk on one instance	EFS mounted by all tasks; AWS Backup
Single API instance	Compose and Render run one API container	At least two ECS tasks in two AZs
Migrations at start	A failed migration stops every starting instance (no rollback of the release)	Deploy migrations with one task first; test migrations on a UAT copy of production; keep the previous image and a pre-deployment snapshot for rollback
Secrets	<code>DATA_ENCRYPTION_KEY</code> loss makes two-factor secrets unreadable; <code>JWT_SECRET</code> change signs everyone out	Secrets Manager with versioning; sealed escrow copy (document 09)
Region	Single region ap-southeast-1	Cross-region snapshot and EFS backup copies; documented regional recovery (document 08)
SMTP provider	One <code>SMTP_URL</code>	Outbox retries; a second provider can be configured by changing <code>SMTP_URL</code>
CloudFront / S3 pipeline	Deployment by GitHub Actions to one bucket	S3 versioning enables rollback to the previous build; the workflow can be re-run on a previous commit

5 Known limitations

Limitation	Impact and recommendation
Per-instance rate limiting	Sign-in and API rate limits are kept in memory per process. With N instances the effective limit is N times the configured value, and counters reset when an instance restarts. Recommended: move the counters to a shared store (PostgreSQL or ElastiCache Redis) or enforce limits in AWS WAF rate-based rules.
Per-instance settings cache	A configuration change is seen by other instances only after restart (document 05). Recommended: cache time-to-live or invalidation by <code>LISTEN / NOTIFY</code> ; until then restart all tasks after configuration changes (a rolling restart has no downtime).
Schedule changes	Editing a scheduled job reloads the cron tasks only on the instance that handled the request. Recommended: restart the tasks or reload schedules periodically.
Scheduler time zone	Cron expressions are evaluated in UTC (container time zone), see document 01.
Concurrent migrations	Two instances starting together with a pending migration may both try to apply it; one fails and restarts. Recommended: advisory lock around <code>migrate()</code> or single-task deployments of schema changes.
Long-running requests	Large report files and imports run inside the web request on the event loop; during that time the instance serves other requests slowly (document 06).
Local file storage API	Files are on a shared file system rather than in S3; EFS gives multi-AZ durability. Replacing the storage functions with S3 would remove the file system dependency (later change).
No automatic retry of failed jobs	A failed job run is recorded (<code>job_runs.status = failed</code>) and runs again at its next schedule; no immediate retry and no alert (document 11).

6 Related documents

This document is one of eleven that together form the BrokerVerse Solution Architecture. They are maintained and issued together (source and build: [docs/architecture/README.md](#)).

No.	Document	File
01	Solution Component Diagram and Application Architecture	01_BrokerVerse_Solution_Component_Diagram_Application_Architecture.pdf
02	Database Design	02_BrokerVerse_Database_Design.pdf
03	Database Inventory	03_BrokerVerse_Database_Inventory.pdf
04	Technology Stack	04_BrokerVerse_Technology_Stack.pdf
05	Shared Service Components	05_BrokerVerse_Shared_Service_Components.pdf
06	Capacity and Performance	06_BrokerVerse_Capacity_and_Performance.pdf
07	High Availability and Resiliency (this document)	07_BrokerVerse_High_Availability_and_Resiliency.pdf
08	RTO and RPO	08_BrokerVerse_RTO_and_RPO.pdf
09	Backup and Recovery	09_BrokerVerse_Backup_and_Recovery.pdf
10	Data Archival, Housekeeping and Restoration	10_BrokerVerse_Data_Archival_Housekeeping_and_Restoration.pdf
11	Monitoring	11_BrokerVerse_Monitoring.pdf

7 Glossary

Terms and abbreviations used in this document.

Term	Meaning
Advisory lock	PostgreSQL application-level lock (<code>pg_try_advisory_lock</code>) used by the scheduler so a job runs on one API instance at a time.
ALB	Application Load Balancer (AWS): distributes HTTPS requests over the API containers and checks their health.
API	Application programming interface; here the BrokerVerse REST API under <code>/api</code> served by the Node.js backend.
AZ	Availability Zone: an isolated data-centre group inside an AWS region.
CloudFront	AWS content delivery network that serves the single-page application and can route <code>/api</code> to the backend.
Cron	Time-based schedule expression (minute hour day month weekday) used by the scheduled jobs.
ECS / Fargate	Elastic Container Service (AWS) and its serverless compute mode for running the API containers.
EFS	Elastic File System (AWS): shared, multi-AZ network file system proposed for the upload directory.
JWT	JSON Web Token: signed token carrying the user id, roles, permissions and token version.
Migration	Numbered SQL file in <code>backend/src/db/migrations</code> applied once, in order, on API start.
Multi-AZ	Deployment with a synchronous standby in a second Availability Zone and automatic failover (RDS).
PITR	Point-in-time recovery: restoring a database to any second inside the backup retention window.
RDS	Amazon Relational Database Service, the managed PostgreSQL target.
Receivable / bill	Premium bill (INV-) raised when a policy, endorsement or renewal is issued.
Remittance	Payment of collected premium, net of commission, from the broker to the insurer.
RPO	Recovery point objective: the maximum acceptable data loss, measured in time.
RTO	Recovery time objective: the maximum acceptable time to restore service.
S3	Amazon Simple Storage Service: object storage hosting the SPA build (and proposed archive store).
Seed	Idempotent data load run on every API start: reference data always, sample data only when <code>SEED_SAMPLE_DATA</code> is on.
SES	Amazon Simple Email Service, a possible SMTP relay.
SMTP	Simple Mail Transfer Protocol; the API sends e-mail through the server in <code>SMTP_URL</code> .
SPA	Single-page application: the React front end served as static files.
UAT	User acceptance testing environment.
WAF	Web Application Firewall.